

Microsoft Fabric Warehouse

Performance Tuning & Optimization

DBA Technical Reference | Volume 4 of 9

April 2025 | Version 1.0

Table of Contents

1.	Performance Fundamentals	3
2.	Statistics Management	3
3.	Table Design Best Practices	4
4.	Data Loading Optimization	5
5.	Query Writing Best Practices	6
6.	Data Compaction Management	7
7.	Caching & Result Reuse	7
8.	Identifying Slow Queries	8
9.	Performance Diagnostics Scripts	9
10.	Performance Tuning Checklist	10

1. Performance Fundamentals

Performance tuning in Fabric Warehouse differs significantly from traditional SQL Server tuning. Since Fabric uses a distributed columnar storage engine built on Parquet and Delta Lake, the levers available to a DBA are different — but equally important.

SQL Server Tuning Lever	Fabric Equivalent / Status
Clustered / Non-clustered indexes	Not available — use statistics and partitioning
Columnstore indexes	Built-in — all storage is columnar Parquet
MAXDOP / parallelism hints	Engine-managed automatically
TempDB optimization	Not applicable — no TempDB in Fabric
Statistics	Available and critical — must be managed proactively
Query Store	Not available — use Query Insights DMVs
Data types & schema design	Critical — same importance as traditional DW
Batch vs row-mode execution	Vectorized execution enabled automatically
Partitioning	Supported — reduces data scanned per query

2. Statistics Management

Statistics are metadata objects that describe the distribution of data values in a column. The query optimizer uses statistics to estimate cardinality and choose the most efficient execution plan. In Fabric Warehouse, statistics are crucial for good query performance.

Statistics are automatically created when:

- A column is first queried with a filter or JOIN predicate
- A column appears in an ORDER BY clause
- Data is loaded using COPY INTO or INSERT statements

>> Create Statistics Manually on a Column

```
-- Create statistics on a high-cardinality filter column
CREATE STATISTICS stat_FactSales_OrderDate
ON dbo.FactSales (OrderDate);

-- Create statistics on a JOIN key column
CREATE STATISTICS stat_FactSales_CustomerKey
ON dbo.FactSales (CustomerKey);
```

>> Create Multi-Column Statistics

```
-- Useful when columns are used together in filters
CREATE STATISTICS stat_FactSales_Region_Date
ON dbo.FactSales (SalesRegion, OrderDate);
```

```
>> Update Stale Statistics

-- Update statistics after significant data loads (>20% row change)
UPDATE STATISTICS dbo.FactSales;

-- Update specific statistic
UPDATE STATISTICS dbo.FactSales stat_FactSales_OrderDate;
```

```
>> View All Statistics on a Table

SELECT
    s.name AS stat_name,
    c.name AS column_name,
    s.auto_created,
    s.user_created,
    STATS_DATE(s.object_id, s.stats_id) AS last_updated
FROM sys.stats s
JOIN sys.stats_columns sc ON s.object_id = sc.object_id
AND s.stats_id = sc.stats_id
JOIN sys.columns c ON sc.object_id = c.object_id
AND sc.column_id = c.column_id
WHERE s.object_id = OBJECT_ID('dbo.FactSales')
ORDER BY s.name;
```

TIP: After any large data load (millions of rows), always run UPDATE STATISTICS on the affected tables before executing production analytical queries.

3. Table Design Best Practices

3.1 Choose Correct Data Types

Scenario	Recommended Type	Avoid
Date values only	DATE	DATETIME, NVARCHAR
Date + Time	DATETIME2(0–7)	DATETIME (less precision)
Integer keys	INT or BIGINT	NVARCHAR, FLOAT
Decimal amounts	DECIMAL(18,2)	FLOAT (rounding errors)
Short text codes	VARCHAR(n)	NVARCHAR if ASCII-only
Long descriptions	NVARCHAR(MAX)	Use sparingly — impacts perf
Boolean flags	BIT	CHAR(1) or TINYINT

3.2 Schema Design Patterns

- Use star schema (fact + dimension tables) for optimal analytical query performance
- Keep fact tables narrow — only foreign keys and numeric measures; push attributes to dimensions

- Use surrogate integer keys (INT/BIGINT) as join keys — avoid NVARCHAR or composite keys
- Separate slowly changing dimensions (SCD Type 2) to avoid large table updates
- Partition large fact tables by date column to enable partition elimination
- Avoid wide tables with hundreds of columns — split into subject-area tables

4. Data Loading Optimization

4.1 COPY INTO Command (Preferred for Bulk Loads)

>> Bulk Load from Azure Data Lake Storage

```
COPY INTO dbo.FactSales
(OrderDate, CustomerKey, ProductKey, SalesAmount, Quantity)
FROM 'https://dfs.core.windows.net/sales/*.parquet'
WITH (
FILE_TYPE = 'PARQUET',
CREDENTIAL = (IDENTITY = 'Managed Identity')
);
```

>> Bulk Load from CSV with Options

```
COPY INTO dbo.StagingOrders
FROM 'https://blob.core.windows.net/raw/orders/*.csv'
WITH (
FILE_TYPE = 'CSV',
FIRSTROW = 2, -- skip header row
FIELDTERMINATOR = ',',
ROWTERMINATOR = '0x0A',
CREDENTIAL = (IDENTITY = 'Managed Identity')
);
```

4.2 Batch INSERT Optimization

>> Insert in Batches (Avoid Row-by-Row)

```
-- Good: Insert from a SELECT (set-based)
INSERT INTO dbo.FactSales
SELECT OrderDate, CustomerKey, ProductKey, SalesAmount
FROM dbo.StagingFactSales
WHERE LoadDate = CAST(GETUTCDATE() AS DATE);

-- Avoid: Row-by-row cursor or application-side loops
-- These generate thousands of small Parquet files = poor read perf
```

Load Pattern	Performance	Notes
COPY INTO (Parquet)	Excellent	Best option — native columnar format
COPY INTO (CSV)	Good	Efficient for bulk loads from raw files
INSERT ... SELECT	Good	Set-based — generates reasonable file sizes
Dataflow Gen2	Good	Power Query transformations written to table
Single-row INSERTs	Poor	Creates many tiny Parquet files — avoid
BULK INSERT	Not supported	Use COPY INTO instead

5. Query Writing Best Practices

>> Use Column Pruning (Select Only What You Need)

```
-- Good: Select only needed columns (columnar storage rewards this)
SELECT CustomerKey, SalesAmount, OrderDate
FROM dbo.FactSales
WHERE OrderDate >= '2024-01-01';

-- Avoid: SELECT * scans all columns from all Parquet files
SELECT * FROM dbo.FactSales WHERE OrderDate >= '2024-01-01';
```

>> Apply Filters Early with CTEs

```
-- Good: Filter in CTE before joining
WITH RecentSales AS (
  SELECT CustomerKey, SalesAmount
  FROM dbo.FactSales
  WHERE OrderDate >= DATEADD(YEAR, -1, GETUTCDATE())
)
SELECT c.CustomerName, SUM(rs.SalesAmount) AS TotalSales
FROM RecentSales rs
JOIN dbo.DimCustomer c ON rs.CustomerKey = c.CustomerKey
GROUP BY c.CustomerName;
```

>> Avoid Functions on Filtered Columns

```
-- Avoid: Function on column prevents predicate pushdown
WHERE YEAR(OrderDate) = 2024

-- Good: Range filter – allows Parquet row group elimination
WHERE OrderDate >= '2024-01-01' AND OrderDate < '2025-01-01'
```

>> Use APPROX_COUNT_DISTINCT for Large Cardinality Estimates

```
-- Faster than COUNT(DISTINCT ...) for large tables
SELECT
  APPROX_COUNT_DISTINCT(CustomerKey) AS approx_customers,
  APPROX_COUNT_DISTINCT(ProductKey) AS approx_products
FROM dbo.FactSales;
```

6. Data Compaction Management

Fabric Warehouse automatically runs a background compaction service that merges small Parquet files into larger, more efficient files. This improves read performance over time but can occasionally conflict with user DML operations.

Compaction Scenario	DBA Action
Many small files from row-by-row INSERTS	Redesign ETL to use batch inserts or COPY INTO
Write-write conflict with compaction	Wrap DML in explicit transaction; catch and retry on failure
Slow reads on recently loaded table	Allow compaction time to run before querying; schedule reports accordingly
High CU consumption from compaction	Monitor via Capacity Metrics App; schedule large loads during off-peak hours

WARNING: Starting October 2025, data compaction preemption is enabled. Compaction aborts if it detects a user query lock. Design explicit transactions carefully to avoid extended lock periods.

7. Caching & Result Reuse

Fabric Warehouse implements intelligent result and data caching to reduce repeated reads from OneLake storage and improve query response times for common workloads.

- Frequently executed identical queries benefit from result caching — use parameterized queries to maximize cache hits
- Avoid adding GETDATE() or RAND() to queries that would otherwise be cacheable — these bust the cache
- Power BI DirectLake mode leverages Delta Lake caching for near-instant report rendering
- Statistics updates invalidate cached plans — plan post-load statistics updates carefully

8. Identifying Slow Queries

>> Find Top 10 Slowest Queries (Last 7 Days)

```
SELECT TOP 10
  command,
  login_name,
  start_time,
  total_elapsed_time_ms / 1000.0 AS elapsed_sec,
  row_count,
  status
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY, -7, GETUTCDATE())
AND status = 'Succeeded'
ORDER BY total_elapsed_time_ms DESC;
```


>> Find Queries with High Row Scans vs Low Output

```
-- High scan-to-output ratio = potential missing statistics or bad filter
SELECT TOP 20
command,
row_count AS rows_returned,
total_elapsed_time_ms / 1000.0 AS elapsed_sec
FROM queryinsights.exec_requests_history
WHERE status = 'Succeeded'
AND total_elapsed_time_ms > 10000 -- more than 10 seconds
ORDER BY total_elapsed_time_ms DESC;
```

9. Performance Diagnostics Scripts

>> Hourly Query Volume and Avg Duration (Last 7 Days)

```
SELECT
CAST(start_time AS DATE) AS query_date,
DATEPART(HOUR, start_time) AS hour_of_day,
COUNT(*) AS query_count,
AVG(total_elapsed_time_ms)/1000.0 AS avg_elapsed_sec,
MAX(total_elapsed_time_ms)/1000.0 AS max_elapsed_sec,
SUM(CASE WHEN status = 'Failed'
THEN 1 ELSE 0 END) AS failed_count
FROM queryinsights.exec_requests_history
WHERE start_time >= DATEADD(DAY, -7, GETUTCDATE())
GROUP BY CAST(start_time AS DATE),
DATEPART(HOUR, start_time)
ORDER BY query_date, hour_of_day;
```

>> Statistics Health Check

```
-- Find tables with no statistics or very old statistics
SELECT
  t.name AS table_name,
  s.name AS stat_name,
  STATS_DATE(s.object_id, s.stats_id) AS last_updated,
  DATEDIFF(DAY,
    STATS_DATE(s.object_id, s.stats_id),
    GETUTCDATE()) AS days_since_update
FROM sys.stats s
JOIN sys.tables t ON s.object_id = t.object_id
WHERE STATS_DATE(s.object_id, s.stats_id) < DATEADD(DAY, -7, GETUTCDATE())
OR STATS_DATE(s.object_id, s.stats_id) IS NULL
ORDER BY days_since_update DESC;
```

10. Performance Tuning Checklist

Area	Checklist Item	Priority
Schema	Use INT/BIGINT surrogate keys on fact tables	High
Schema	Use DATE not DATETIME for date-only columns	High
Schema	Avoid VARCHAR(MAX) / NVARCHAR(MAX) unless necessary	Medium
Statistics	Create statistics on all JOIN keys	High
Statistics	Create statistics on high-cardinality filter columns	High
Statistics	Run UPDATE STATISTICS after large loads (>20% change)	High
Loading	Use COPY INTO for bulk loads instead of row-by-row INSERT	High
Loading	Batch INSERTs using INSERT...SELECT rather than loops	High
Queries	Avoid SELECT * — specify only needed columns	Medium
Queries	Apply date range filters as literals, not function-wrapped	High
Queries	Use CTEs to filter early before joins	Medium
Monitoring	Review long_running_queries weekly	High
Monitoring	Check statistics age after major data loads	High
Monitoring	Monitor CU consumption via Capacity Metrics App	Medium